TP Projet 2 : Investment Agent et MDM Produits

HESTIM | Data Governance | Projet "Enterprise Multi-Agent AI Governance Platform"
Membre : Investment | Dataset : S&P 500 Historical Market Data | Prérequis : TP0 terminé

Objectifs pédagogiques

À l'issue de ce TP, vous serez capables de :

- Inspecter un gros dataset (plusieurs millions de lignes) sans le charger en entier
- Charger un dataset brut dans PostgreSQL avec Python, par morceaux
- Documenter un dataset avec les 4 types de métadonnées (fiche catalogue)
- Rédiger un dictionnaire de données selon les règles du cours
- Cataloguer et classifier un dataset dans OpenMetadata (owner, domaine, tags, glossaire)
- Appliquer les 5 tests du MDM et distinguer **Master Data** et **données transactionnelles**
- Fusionner **deux sources** dans un Golden Record `security` avec une règle de priorité
- Relier votre référentiel à celui d'un autre membre (le lien ticker → entreprise)

Votre mission

Vous êtes le **Data Steward du domaine Investissement**. L'Investment Agent (partie 2) répondra à des questions comme "*Quelle est la volatilité de ce titre sur 12 mois ?*". Pour cela, il faut :

1. que l'historique des cours soit chargé et **documenté** ;
2. que chaque titre boursier existe **une seule fois**, sous une forme fiable (le **Golden Record**) ;
3. que chaque titre soit relié à **l'entreprise** qui l'émet, dans le référentiel du membre Finance.

Votre MDM s'appelle **Produits** (domaine **Things** du cours) :

- **Security** : un titre boursier (une action cotée, identifiée par son ticker). Données réelles.
- **Sector** : le secteur GICS (Reference Data). Données réelles.
- **Product** et **Investor** : les produits d'investissement de notre entreprise fictive et ses clients investisseurs. Petites tables **simulées** (une dizaine de lignes) pour illustrer le MDM.

Élément	Valeur
Dataset	`sp500_market_data`
Classification	Internal
Sensibilité	Medium
PII	Non
Data Owner	Direction Investissement
Data Steward	Vous
Relecteur de vos PR	Membre HR

Concepts clés pour ce TP

> **DÉFINITION : Ticker et Security**
> Un **ticker** est le code court d'un titre en bourse (ex. `AAPL` pour Apple). La **Security** (titre, instrument) est l'objet financier lui-même. Une entreprise (Company) **émet** une ou plusieurs securities. Exemple : Alphabet émet `GOOGL` et `GOOG`.

> **DÉFINITION : Donnée transactionnelle**
> Une donnée qui décrit un **événement daté**. Un cours de clôture du 12 mars est un événement : il y en a un nouveau chaque jour. C'est l'inverse d'une Master Data. Votre dataset contient surtout des données transactionnelles (les cours), et **quelques** Master Data cachées dedans (les tickers).

> **DÉFINITION : Règle de priorité entre sources**
> Quand deux sources décrivent la même entité, la règle dit **laquelle fait autorité** pour chaque attribut. Ici : la liste officielle des membres du S&P 500 fait autorité sur le nom et le secteur. Le fichier des cours ne sert qu'à **découvrir** des tickers absents de la liste.

ÉTAPE 1 : Découvrir le dataset

1.1 Télécharger

Vous avez besoin de **deux fichiers**, à placer dans `data/raw/sp500/` :

Fichier	Contenu	Où le trouver
Historique des cours	Une ligne par titre et par jour (environ 4,6 millions de lignes)	Le dataset Kaggle "S&P 500 Historical Market Data" choisi par l'équipe
`constituents.csv`	La liste des entreprises du S&P 500 : ticker, nom, secteur GICS, CIK	https://datahub.io/core/s-and-p-500-companies (bouton de téléchargement CSV)

> **EXPLICATION : pourquoi un deuxième fichier ?**
> L'historique des cours donne des tickers et des prix, mais rarement le nom, le secteur ou le CIK. Or le CIK est **la clé** qui permet de relier un titre à l'entreprise du référentiel Finance. `constituents.csv` est notre **deuxième source** : c'est exactement la situation pour laquelle un MDM existe.

1.2 Inspecter les vraies colonnes

Créez `scripts/explore_sp500.py` (remplacez `prices.csv` par le vrai nom de votre fichier) :

```
# Affiche les colonnes et un apercu des deux fichiers S&P 500
import pandas as pd

for f in ["prices.csv", "constituents.csv"]:
    df = pd.read_csv(f"data/raw/sp500/{f}", nrows=5000)
    print(f"\n===== {f} : {len(df.columns)} colonnes")
    print(list(df.columns))
    print(df.head(3).T)
```

```
# Nombre total de lignes du gros fichier, sans tout charger en memoire
with open("data/raw/sp500/prices.csv", encoding="utf-8") as fichier:
    print("\nLignes de prices.csv :", sum(1 for _ in fichier) - 1)
```

```
python scripts/explore_sp500.py
```

Questions à noter dans votre compte rendu :

1. Comment s'appelle la colonne du ticker dans chaque fichier (`Symbol`, `Name`, `Ticker`...) ?
2. Quelles colonnes de prix existe-t-il (`Open`, `High`, `Low`, `Close`, `Adj Close`) ? Que signifie `Adj Close` ?
3. Quelle est la plus ancienne date ? La plus récente ?
4. Cherchez Berkshire Hathaway dans les deux fichiers : le ticker s'écrit-il `BRK.B` ou `BRK-B` ? Que remarquez-vous ?

> **ATTENTION**
> Les noms de colonnes changent d'un dataset Kaggle à l'autre. Dans la suite, on suppose `date`, `symbol`, `open`, `high`, `low`, `close`, `volume` (après mise en minuscules). **Si vos colonnes ont d'autres noms, adaptez** le script de l'étape 2 et le SQL de l'étape 5.

> **CAPTURE D'ÉCRAN REQUISE**
> La sortie du script : colonnes des deux fichiers et nombre de lignes.

ÉTAPE 2 : Charger les données brutes dans PostgreSQL

On charge une **copie brute** dans le schéma `data`, avec le suffixe `_raw` : mêmes colonnes, aucune correction. Les corrections viendront dans le MDM (étape 5).

Installez les dépendances Python (`sqlalchemy` est déjà dans `requirements.txt`) :

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Créez `scripts/load_sp500.py` :

```
# Charge les fichiers S&P 500 bruts dans PostgreSQL (schema data, tables *_raw)
import os
import pandas as pd
from dotenv import load_dotenv
from sqlalchemy import create_engine

load_dotenv()
engine = create_engine(
    f"postgresql+psycopg2://{os.getenv('POSTGRES_USER')}:{os.getenv('POSTGRES_PASSWORD')}"
    f"@localhost:{os.getenv('POSTGRES_PORT')}/{os.getenv('POSTGRES_DB')}"
)
RAW = "data/raw/sp500"
DATE_MIN = None   # ex. "2019-01-01" si le chargement est trop long (a documenter !)
```

```
def nettoyer_noms(df):
    # "Adj Close" -> "adj_close" : noms de colonnes en snake_case
    df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]
    return df


# 1. Liste des membres du S&P 500 (petit fichier)
const = nettoyer_noms(pd.read_csv(f"{RAW}/constituents.csv", dtype=str))
const.to_sql("sp500_constituents_raw", engine, schema="data", if_exists="replace", index=False)
print("sp500_constituents_raw :", len(const))

# 2. Historique des cours (gros fichier) : lu et charge par morceaux
total = 0
for i, chunk in enumerate(pd.read_csv(f"{RAW}/prices.csv", dtype=str, chunksize=200_000)):
    chunk = nettoyer_noms(chunk)
    if DATE_MIN:
        chunk = chunk[chunk["date"] >= DATE_MIN]
    chunk.to_sql("sp500_prices_raw", engine, schema="data",
                 if_exists="replace" if i == 0 else "append", index=False)
    total += len(chunk)
    print("lignes chargees :", total)
```

```
python scripts/load_sp500.py
```

Le chargement peut prendre 10 à 20 minutes. C'est normal pour 4,6 millions de lignes.

Vérification :

```
docker exec gouv_postgres psql -U gouv -d governance -c "SELECT COUNT(*) FROM data.sp500_prices_raw;"
docker exec gouv_postgres psql -U gouv -d governance -c "SELECT COUNT(DISTINCT symbol) FROM
data.sp500_prices_raw;"
```

> **EXPLICATION : pourquoi tout charger en texte (`dtype=str`) ?**
> La table brute doit être une **copie fidèle** du fichier. Si pandas convertit une valeur bizarre (une date mal formée, un prix vide), on perd l'information sans s'en rendre compte. On convertit les types **plus tard**, dans le MDM, là où l'on peut **compter** les valeurs invalides.

> **CAPTURE D'ÉCRAN REQUISE**
> La fin du chargement et les deux requêtes de vérification.

Commit : `TP2 etape 2 : chargement brut du dataset S&P 500`

ÉTAPE 3 : Documenter le dataset (métadonnées et dictionnaire)

3.1 La fiche catalogue (niveau dataset)

Créez `docs/dictionnaire/sp500_market_data.md` et commencez par cette fiche. Complétez les `...`.

Type	Attribut	Valeur
Métier	Nom métier	Historique des cours des actions du S&P 500

Type	Attribut	Valeur
Métier	Description	... (2 phrases : que contient-il, à quoi sert-il)
Métier	Domaine	Investissement
Technique	Tables	`data.sp500_prices_raw`, `data.sp500_constituents_raw`
Technique	Format source	CSV, séparateur virgule
Opérationnelle	Sources	Kaggle (cours), datahub.io (constituents)
Opérationnelle	Période couverte	du ... au ...
Opérationnelle	Volumétrie	... lignes, ... tickers
Opérationnelle	Filtre appliqué	aucun / à partir du ... (si `DATE_MIN` utilisé)
Gouvernance	Data Owner	Direction Investissement
Gouvernance	Data Steward	... (votre nom)
Gouvernance	Classification	Internal
Gouvernance	Sensibilité	Medium
Gouvernance	PII	Non

3.2 Le dictionnaire de données (niveau colonne)

Documentez **au moins 8 colonnes** : `date`, `symbol`, `open`, `high`, `low`, `close`, `adj_close`, `volume`, et dans `constituents` : `security`, `gics_sector`, `cik`.

Exemple rempli :

Nom physique	Nom métier	Définition	Type	Contraintes	Valeurs admises	Règle de gestion	PII
`close`	Cours de clôture	Dernier prix auquel le titre s'est échangé pendant la séance	Décimal (4 décimales)	NOT NULL	Nombre strictement positif, en USD	Doit être compris entre `low` et `high` du même jour	Non
`symbol`	Ticker	Code court qui identifie un titre en bourse	Texte	NOT NULL	Lettres majuscules, tiret pour les classes d'action (`BRK-B`)	Doit exister dans `master.security`	Non

> **EXPLICATION : les 4 règles de rédaction du cours**
>
> 1. **La définition** est une phrase complète, sans jargon technique.
> 2. **La règle de gestion** commence par un verbe et se vérifie ("Doit être compris entre...").
> 3. **Les valeurs admises** sont exhaustives et explicites.
> 4. **Le propriétaire** est toujours une direction métier, jamais l'IT.

Commit : `TP2 etape 3 : fiche catalogue et dictionnaire S&P 500`

ÉTAPE 4 : Cataloguer dans OpenMetadata

4.1 Ingestion

1. **Paramètres, Services, Bases de données**, ouvrir `gouv_postgres` (créé au TP0).
2. Onglet **Ingestions** (ou **Agents** selon la version), **Ajouter une ingestion de métadonnées**.
3. Filtre de schémas : inclure `data`, `master`, `ref`.
4. Lancer l'ingestion et attendre le statut **Success**.
5. Dans **Explorer**, chercher `sp500_prices_raw`.

> **EXPLICATION : ce que l'ingestion a deviné toute seule**
> Les métadonnées **techniques** (colonnes, types) et **opérationnelles** (nombre de lignes). Les métadonnées **métier** et **de gouvernance**, c'est votre travail de Steward.

4.2 Enrichir les 2 tables brutes

Champ OpenMetadata	Valeur
Description	Reprendre votre fiche catalogue
Owner	Équipe `Direction Investissement`
Domaine	`Investissement`
Tags	`Confidentialite.Internal`, `Sensibilite.Medium`

Sur 3 ou 4 colonnes (`symbol`, `close`, `volume`...), ajoutez la description du dictionnaire et le tag `PII.NonSensitive`.

4.3 Glossaire

Dans `Glossaire Projet`, créez :

Terme	Définition	Lier à
Security	Titre financier coté, émis par une entreprise	table `master.security` (étape 5)
Ticker	Code court qui identifie un titre en bourse	colonne `symbol`
Cours de clôture	Dernier prix échangé pendant une séance	colonne `close`
Volatilité	Mesure de l'ampleur des variations d'un cours sur une période	(sera calculée par l'agent, partie 2)

> **CAPTURE D'ÉCRAN REQUISE**
> La page de `sp500_prices_raw` avec owner, domaine, tags et description. Le glossaire avec vos 4 termes.

ÉTAPE 5 : Construire le MDM Produits

5.1 Choisir les entités maîtres avec les 5 tests

Complétez dans votre compte rendu (OUI / NON + justification courte) :

Entité	Nature	Volatilité faible	Réutilisation	Partage	Criticité	Master Data ?
Security (ticker)	...	...	...	...	...	...
Sector GICS	...	...	...	...	...	...
Un cours du jour (une ligne de prices)	...	...	...	...	...	...
Product (fonds d'investissement)	...	...	...	...	...	...

> **EXPLICATION : le résultat attendu**
> Security et Sector sont des **entités** stables, utilisées par des milliers de lignes de cours : Master Data. Un cours du jour est un **événement** qui change chaque jour : donnée transactionnelle. Il ira dans `data.market_prices` et **pointera** vers `master.security`.

5.2 Le modèle

```
   ref.sector (GICS)            master.company  <- referentiel du membre Finance
          \                          /
           \                        / (par le CIK)
            +--- master.security -+
                 (Golden Record)
                       |
              data.market_prices          master.product   master.investor
            (un cours = une ligne)        (simule)         (simule)
```

5.3 Les tables simulées

Créez `data/seed/product.csv` (ces données sont fictives, elles peuvent aller dans Git) :

```
code,label,category,risk_level
FND-US-EQ,Fonds Actions US,Equity Fund,High
FND-US-TECH,Fonds Technologie US,Equity Fund,High
FND-DIV,Fonds Dividendes,Equity Fund,Medium
FND-BAL,Fonds Equilibre,Balanced Fund,Medium
MAND-PRUD,Mandat Prudent,Discretionary Mandate,Low
MAND-DYN,Mandat Dynamique,Discretionary Mandate,High
```

Créez `data/seed/investor.csv` :

```
code,name,investor_type,country,risk_profile
INV-001,Caisse de Retraite Atlas,Pension Fund,MA,Low
INV-002,Assurances Horizon,Insurance,MA,Medium
INV-003,Banque Meridien,Bank,FR,Medium
INV-004,Family Office Cedre,Family Office,MA,High
INV-005,Fondation Oasis,Foundation,MA,Low
```

> **REMARQUE : pourquoi des investisseurs institutionnels ?**
> Ce sont des **personnes morales**, donc il n'y a **pas de PII**. Si nous avions mis des clients particuliers (nom, prénom, âge), la table deviendrait **Confidential** avec des PII, comme le dataset RH. Choisir le bon niveau de détail fait partie de la gouvernance : on ne collecte que ce dont on a besoin.

5.4 Le script MDM

Créez `sql/21_investment_mdm.sql` :

```
-- MDM Produits (membre Investment)
-- Pre-requis : data.sp500_prices_raw, data.sp500_constituents_raw charges
--              master.company publie par le membre Finance (sinon company_id reste NULL)

-- 1. Reference Data : secteurs GICS ----------------------------------
CREATE TABLE IF NOT EXISTS ref.sector (
  sector_id SERIAL PRIMARY KEY,
  label     VARCHAR(80) UNIQUE NOT NULL);
INSERT INTO ref.sector (label)
SELECT DISTINCT TRIM(gics_sector) FROM data.sp500_constituents_raw
WHERE gics_sector IS NOT NULL
ON CONFLICT DO NOTHING;

-- 2. Golden Record Security ------------------------------------------
CREATE TABLE IF NOT EXISTS master.security (
  security_id SERIAL PRIMARY KEY,
  ticker      VARCHAR(12) UNIQUE NOT NULL,
  name        VARCHAR(200),
  sector_id   INT REFERENCES ref.sector(sector_id),
  cik         VARCHAR(10),
  company_id  INT,                  -- lien vers master.company (Finance)
  in_index    BOOLEAN NOT NULL,     -- membre actuel du S&P 500 ?
  source      VARCHAR(20) NOT NULL, -- d'ou vient la fiche
  updated_at  TIMESTAMP DEFAULT now());

-- Standardisation du ticker : majuscules, sans espaces, '.' remplace par '-'
-- (BRK.B et BRK-B sont le MEME titre)

-- Source 1 (prioritaire) : la liste officielle du S&P 500
INSERT INTO master.security (ticker, name, sector_id, cik, in_index, source)
SELECT REPLACE(UPPER(TRIM(c.symbol)), '.', '-'),
       TRIM(c.security),
       s.sector_id,
       LPAD(c.cik, 10, '0'),
       TRUE,
       'CONSTITUENTS'
FROM data.sp500_constituents_raw c
LEFT JOIN ref.sector s ON s.label = TRIM(c.gics_sector)
ON CONFLICT (ticker) DO UPDATE
  SET name = EXCLUDED.name, sector_id = EXCLUDED.sector_id,
      cik = EXCLUDED.cik, in_index = TRUE, source = 'CONSTITUENTS',
      updated_at = now();

-- Source 2 (secondaire) : tickers vus dans les cours mais absents de la liste
-- (titres sortis de l'indice). Regle de priorite : on ne cree que ce qui manque.
INSERT INTO master.security (ticker, in_index, source)
SELECT DISTINCT REPLACE(UPPER(TRIM(symbol)), '.', '-'), FALSE, 'PRICES'
FROM data.sp500_prices_raw
WHERE symbol IS NOT NULL
ON CONFLICT (ticker) DO NOTHING;

-- Lien vers le referentiel Finance, par le CIK (seulement si master.company existe)
DO $$
BEGIN
  IF to_regclass('master.company') IS NOT NULL THEN
    UPDATE master.security sec
```

```
    SET company_id = c.company_id
    FROM master.company c
    WHERE c.cik = sec.cik;
  END IF;
END $$;

-- 3. Cours (donnees metier qui pointent vers le MDM) -----------------------
CREATE TABLE IF NOT EXISTS data.market_prices (
  price_id    BIGSERIAL PRIMARY KEY,
  security_id INT NOT NULL REFERENCES master.security(security_id),
  trade_date  DATE NOT NULL,
  open        NUMERIC(14,4),
  high        NUMERIC(14,4),
  low         NUMERIC(14,4),
  close       NUMERIC(14,4),
  volume      BIGINT);

TRUNCATE data.market_prices;
INSERT INTO data.market_prices (security_id, trade_date, open, high, low, close, volume)
SELECT sec.security_id, p.date::DATE,
       NULLIF(p.open,'')::NUMERIC, NULLIF(p.high,'')::NUMERIC,
       NULLIF(p.low,'')::NUMERIC,  NULLIF(p.close,'')::NUMERIC,
       NULLIF(p.volume,'')::NUMERIC::BIGINT
FROM data.sp500_prices_raw p
JOIN master.security sec ON sec.ticker = REPLACE(UPPER(TRIM(p.symbol)), '.', '-');

-- 4. Tables simulees -------------------------------------------------
CREATE TABLE IF NOT EXISTS master.product (
  product_id SERIAL PRIMARY KEY,
  code       VARCHAR(20) UNIQUE NOT NULL,
  label      VARCHAR(100) NOT NULL,
  category   VARCHAR(40),
  risk_level VARCHAR(8) CHECK (risk_level IN ('Low','Medium','High')));

CREATE TABLE IF NOT EXISTS master.investor (
  investor_id   SERIAL PRIMARY KEY,
  code          VARCHAR(20) UNIQUE NOT NULL,
  name          VARCHAR(150) NOT NULL,
  investor_type VARCHAR(40),
  country       CHAR(2),
  risk_profile  VARCHAR(8) CHECK (risk_profile IN ('Low','Medium','High')));

TRUNCATE master.product, master.investor;
COPY master.product (code, label, category, risk_level)
  FROM '/seed/product.csv' WITH (FORMAT csv, HEADER true);
COPY master.investor (code, name, investor_type, country, risk_profile)
  FROM '/seed/investor.csv' WITH (FORMAT csv, HEADER true);
```

Exécutez-le :

```
docker exec gouv_postgres psql -U gouv -d governance -v ON_ERROR_STOP=1 -f /sql/21_investment_mdm.sql
```

À la deuxième exécution, des messages `NOTICE: ... already exists, skipping` apparaissent : c'est normal, le script est prévu pour être relancé sans danger.

ATTENTION : dépendance avec le membre Finance
Le lien `company_id` n'est rempli que si `master.company` existe dans **votre** base. Si le membre Finance n'a pas encore publié son MDM, ce n'est pas bloquant : `company_id` reste vide. Récupérez sa branche plus tard, rejouez son script, puis relancez le vôtre. C'est pour cela que `company_id` n'est pas `NOT NULL`.

EXPLICATION : les étapes du Golden Record, dans le script

1. **Charger** deux sources : `constituents` (fiable, riche) et `prices` (exhaustive sur les tickers).
2. **Standardiser** le ticker : `REPLACE(UPPER(TRIM(...)), '.', '-')`, donc `brk.b`, `BRK.B` et `BRK-B` donnent tous `BRK-B`.
3. **Dédoublonner** : `ticker UNIQUE` interdit deux fiches pour le même titre.
4. **Appliquer la règle de priorité** : `constituents` écrase (`DO UPDATE`), `prices` ne fait que compléter (`DO NOTHING`).
5. **Publier** `master.security` : les cours pointent vers lui par `security_id`, et lui pointe vers `master.company`.

5.5 Contrôles qualité du référentiel

Créez `sql/22_investment_quality.sql` et exécutez-le de la même façon :

```
-- Q1. Unicite : un ticker n'apparait jamais deux fois (attendu : 0 ligne)
SELECT ticker, COUNT(*) FROM master.security GROUP BY ticker HAVING COUNT(*) > 1;

-- Q2. Pourquoi standardiser ? Tickers bruts qui ne different que par '.' ou '-'
SELECT REPLACE(UPPER(TRIM(symbol)), '.', '-') AS ticker_std,
       ARRAY_AGG(DISTINCT symbol) AS variantes
FROM data.sp500_prices_raw
GROUP BY 1 HAVING COUNT(DISTINCT symbol) > 1;

-- Q3. Completude du Golden Record (seuil : 95 %)
SELECT source, COUNT(*) AS nb,
       ROUND(100.0 * COUNT(name)      / COUNT(*), 2) AS pct_name,
       ROUND(100.0 * COUNT(sector_id) / COUNT(*), 2) AS pct_sector
FROM master.security GROUP BY source;

-- Q4. Lien inter-referentiels : part des titres relies a une entreprise Finance
SELECT ROUND(100.0 * COUNT(company_id) / COUNT(*), 2) AS pct_lie_company
FROM master.security WHERE in_index;

-- Q5. Doublons de cours (meme titre, meme jour)
SELECT security_id, trade_date, COUNT(*) FROM data.market_prices
GROUP BY 1, 2 HAVING COUNT(*) > 1 LIMIT 10;

-- Q6. Validite des cours : prix negatifs ou incoherents
SELECT COUNT(*) FILTER (WHERE close <= 0)  AS close_negatif,
       COUNT(*) FILTER (WHERE high < low)  AS high_inf_low,
       COUNT(*) FILTER (WHERE close IS NULL) AS close_vide
FROM data.market_prices;
```

Ce que vous devez faire avec les résultats (c'est le travail du Steward) :

- **Q3** : les fiches `source = 'PRICES'` n'ont ni nom ni secteur. Expliquez pourquoi, et dites si c'est acceptable (ce sont des titres sortis de l'indice).

- **Q4** : un taux faible est normal si Finance n'a chargé qu'un trimestre (toutes les entreprises n'ont pas déposé de 10-K ce trimestre-là). Discutez-en avec le membre Finance et notez votre conclusion commune.
- **Q5 ou Q6** renvoient des lignes ? Notez combien, et proposez une règle de nettoyage (sans l'appliquer aux tables brutes).

> **CAPTURE D'ÉCRAN REQUISE**
> Les résultats de Q1 à Q4, et `SELECT * FROM master.security LIMIT 10;`.

5.6 Mettre à jour le catalogue

1. Relancez l'ingestion OpenMetadata : `master.security`, `master.product`, `master.investor`, `ref.sector`, `data.market_prices` apparaissent.
2. Documentez `master.security` : description "*Golden Record des titres : une ligne par ticker standardisé, source prioritaire : liste officielle S&P 500*", owner `Direction Investissement`, domaine `Investissement`, tags `Confidentialite.Internal`, `Sensibilite.Low`.
3. Documentez `master.product` et `master.investor` en précisant **"données simulées"** dans la description.
4. Liez le terme **Security** à `master.security`.
5. (Optionnel) Onglet **Lignage** : `data.sp500_constituents_raw` et `data.sp500_prices_raw` → `master.security` → `data.market_prices`.

Commit : `TP2 etape 5 : MDM Produits et controles qualite` Puis ouvrez une **Pull Request** vers `main`, relue par le membre HR.

Questions de compréhension (à préparer pour l'oral)

1. Pourquoi un cours de bourse n'est-il pas une Master Data alors que le ticker en est une ?
2. Quelle est votre règle de priorité entre les deux sources ? Pourquoi celle-là ?
3. Que se passerait-il sans la standardisation `.` → `-` ? Donnez l'exemple trouvé en Q2.
4. Comment votre référentiel est-il relié à celui du membre Finance ? Que se passe-t-il si Finance change le format du CIK ?
5. Pourquoi `master.investor` n'a-t-elle pas de PII ? Que changerait l'ajout de clients particuliers ?
6. Quelles métadonnées OpenMetadata a-t-il trouvées seul, et lesquelles avez-vous ajoutées ?

Livrables

- [ ] `scripts/explore_sp500.py`, `scripts/load_sp500.py`
- [ ] `docs/dictionnaire/sp500_market_data.md` (fiche + 8 colonnes minimum)
- [ ] `data/seed/product.csv`, `data/seed/investor.csv`
- [ ] `sql/21_investment_mdm.sql`, `sql/22_investment_quality.sql`
- [ ] Tableau des 5 tests rempli
- [ ] Captures d'écran des étapes 1, 2, 4 et 5
- [ ] Pull Request relue par le membre HR

Suite (partie 2) : tables de gouvernance, Governance Engine, Investment Agent (dont la règle ABAC : lire SEC seulement si `purpose = valuation`).